

CSE 220 PROJECT

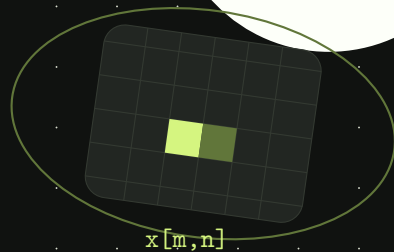
Image Lab

2D convolution, resampling,
anti-aliasing, metrics

theory → NumPy implementation → visual
output

Team members: _____

Course teacher: _____

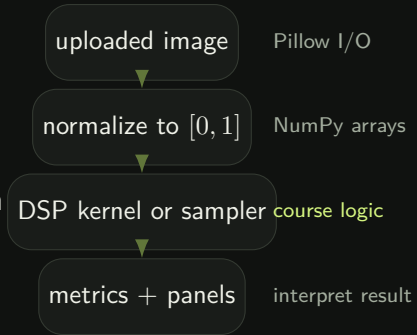


What the project currently demonstrates

A digital image is treated as a 2D discrete signal

$x[m, n] \in [0, 1]$, $m = \text{row}, n = \text{column}$

- ▶ Convolution with editable impulse responses / kernels
- ▶ Blur, sharpen, Laplacian, Sobel and emboss presets
- ▶ Boundary handling: reflect, edge, wrap, zero
- ▶ Nearest and bilinear resizing



Core design: keep math separate from web code

Design principle

The course-relevant algorithms live in `dsp_utils.py`. Django only loads images, routes requests and displays results. This makes the DSP logic testable like a normal NumPy module.

`imaging.py`

`operations.py`

`dsp_utils.py`

```
1 def to_float(image):
2     array = np.asarray(image)
3     if array.dtype == np.uint8:
4         return array.astype(np.float64)
5         / 255.0
6     return np.clip(array.astype(np.
7 float64), 0.0, 1.0)
8
9 def _per_channel(func, image, *args, **
10 kwargs):
11     image = np.asarray(image)
12     if image.ndim == 2:
13         return func(image, *args, **
14 kwargs)
15     return np.stack([
16         func(image[..., c], *args, **
17 kwargs)
18         for c in range(image.shape[-1])
19     ], axis=-1)
```

Feature 1: 2D convolution

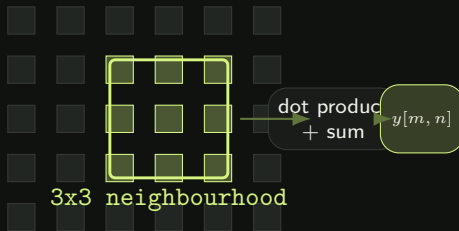
Theory

For an image $x[m, n]$ and filter/kernel $h[m, n]$:

$$y[m, n] = \sum_i \sum_j x[i, j] h[m-i, n-j]$$

The kernel is the impulse response of a 2D LTI system.

- Blur averages nearby samples.
- Sharpen boosts high-frequency detail.
- Sobel/Laplacian preserve edges.



How convolution is implemented

```
1 def _convolve2d_plane(plane, kernel,
2   pad_mode="reflect"):
3   kh, kw = kernel.shape
4   top, bottom, left, right =
5   _anchor_padding(kh, kw)
6   flipped = kernel[::-1, ::-1]
7
8   padded = np.pad(plane, ((top,
9   bottom),(left, right)),
10    mode=np_mode)
11   windows = sliding_window_view(
12   padded, (kh, kw))
13
14   return np.einsum("mnuv,uv->mn",
15    windows, flipped,
16    optimize=True)
```

Key implementation techniques

- ▶ Flip kernel: true convolution, not correlation.
- ▶ Create all neighbourhoods with `sliding_window_view`.
- ▶ Use `np.einsum` for the double summation.
- ▶ Run the same function channel-by-channel for RGB.

Feature 2: kernels as different filters

Blur

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Low-pass: smooths variation.

Sharpen

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Boosts edge/detail energy.

Sobel X

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Derivative-like edge detector.

Unifying idea: the code does not need separate algorithms for blur, sharpen or edge detection. The operation is always convolution; only the kernel coefficients change.

Kernel presets and normalization

```
1 KERNEL_PRESETS = {
2     "box_blur_3": [[1/9, 1/9, 1/9],
3                   [1/9, 1/9, 1/9],
4                   [1/9, 1/9, 1/9]],
5     "sharpen":
6     [[0, -1, 0], [-1, 5, -1], [0, -1, 0]],
7     "sobel_x":
8     [[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]],
9 }
10
11 def normalize_kernel(kernel):
12     total = kernel.sum()
13     if np.isclose(total, 0.0):
14         return kernel
15     return kernel / total
```

DC gain

For a constant image $x[m, n] = C$,

$$y[m, n] = C \sum_i \sum_j h[i, j]$$

- ▶ $\sum h = 1$: brightness is preserved.
- ▶ $\sum h = 0$: DC is removed; edges remain.
- ▶ Zero-sum kernels must not be normalized by division.

Feature 3: boundary handling

Problem

A kernel centered at an edge asks for pixels outside the image. The project exposes four signal assumptions:

- ▶ **reflect** mirror the boundary
- ▶ **edge** repeat the edge value
- ▶ **wrap** periodic/circular extension
- ▶ **zero** assume outside is zero



Signal connection: wrap mode treats the image like a periodic signal, similar to circular

Feature 4: resizing from sample coordinates

Mapping output pixels back to input

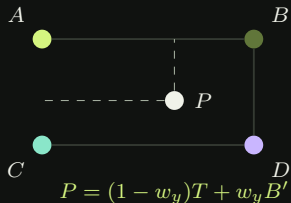
$$s = (d + 0.5) \frac{N_{in}}{N_{out}} - 0.5$$

The 0.5 terms align pixel centers, keeping the image geometrically centered.

```
1 def _sample_coords(in_size, out_size):
2     return ((np.arange(out_size) + 0.5)
3             * (in_size / out_size) -
4             0.5)
5
6 def resize_nearest(image, out_h, out_w):
7     ys = np rint(_sample_coords(in_h,
8     out_h))
9     xs = np rint(_sample_coords(in_w,
10    out_w))
11    return image[ys[:, None], xs[None,
12    :]]
```

- Nearest: choose closest sample
- Bilinear: blend four surrounding samples

Bilinear interpolation



$$T = (1 - w_x)A + w_x B$$

$$B' = (1 - w_x)C + w_x D$$

$$P = (1 - w_y)T + w_y B'$$

In code, these equations are vectorized over the whole image using NumPy indexing and broadcasting.

Bilinear implementation in code

```
1 wy = (y - y0).reshape(-1, 1)
2 wx = (x - x0).reshape(1, -1)
3
4 top_left      = image[y0c[:,None], x0c[None,:]]
5 top_right     = image[y0c[:,None], x1c[None,:]]
6 bottom_left   = image[y1c[:,None], x0c[None,:]]
7 bottom_right  = image[y1c[:,None], x1c[None,:]]
8
9 top = top_left * (1.0 - wx) + top_right * wx
10 bottom = bottom_left * (1.0 - wx) + bottom_right * wx
11 return top * (1.0 - wy) + bottom * wy
```

What makes this efficient

- ▶ No Python loop over every pixel.
- ▶ Index arrays fetch all four neighbor grids at once.
- ▶ Broadcasting applies w_x and w_y across rows, columns and RGB channels.

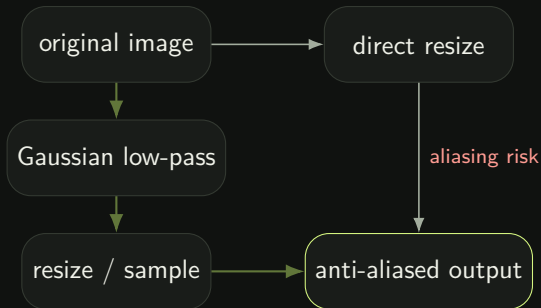
Feature 5: anti-aliasing before downsampling

Sampling idea

Downsampling lowers the sampling rate. High spatial frequencies above the new Nyquist limit fold back as false low-frequency patterns.

$$D = \frac{N_{in}}{N_{out}}, \quad \omega_{new} = \frac{\pi}{D}$$

Fix: low-pass filter first, then keep fewer samples.



Anti-aliasing implementation

```
1 def antialias_sigma(in_size, out_size):
2     decimation = in_size / float(
3         out_size)
4     if decimation <= 1.0:
5         return 0.0
6     return (decimation - 1.0) / 2.0
7
8 if antialias:
9     sigma_y = antialias_sigma(in_h,
10                                out_h)
11     sigma_x = antialias_sigma(in_w,
12                                out_w)
13     work = convolve_separable(
14         work,
15         gaussian_kernel1d(sigma_x),
16         gaussian_kernel1d(sigma_y),
17         pad_mode)
```

Why separable Gaussian?

A 2D Gaussian can be factored:

$$G(x, y) = G_x(x)G_y(y)$$

So filtering is done in two 1D passes:

$$O(k^2) \rightarrow O(2k)$$

low-pass

separable

vectorized

Feature 6: visual and numerical comparison

```
1 def mse(a, b):
2     return float(np.mean((a - b) ** 2))
3
4 def psnr(a, b, peak=1.0):
5     error = mse(a, b)
6     if error <= 1e-20:
7         return float("inf")
8     return float(10.0 * np.log10((peak
9         **2) / error))
10
11 def difference_map(a, b, gain="auto"):
12     diff = np.abs(a - b)
13     peak = diff.max()
14     gain = 1.0 if peak <= 1e-12 else
15         1.0 / peak
16     return np.clip(diff * gain, 0.0,
17         1.0)
```

Metrics

$$MSE = \frac{1}{N} \sum_i (A_i - B_i)^2$$

$$PSNR = 10 \log_{10} \left(\frac{MAX^2}{MSE} \right)$$

$$D[m, n] = |A[m, n] - B[m, n]|$$

The project does not only show images; it measures how much the result changed.

Implemented operation workflow



Convolution operation

Parses the kernel, chooses padding, optionally normalizes, applies `dsp.convolve2d()`, then reports DC gain, MSE, PSNR and clipping.

Resampling operation

Produces no-AA and AA outputs, enlarges them fairly for comparison, then estimates alias difference and MSE/P-SNR.

Operation registry technique

```
1 OPERATIONS = {}
2
3 def register(op_id, label,
4              description):
5     def decorator(func):
6         OPERATIONS[op_id] =
7         Operation(
8             op_id, label,
9             description, func)
10        return func
11    return decorator
12
13 @register("convolve", "Convolution",
14         "...")
15 def op_convolve(image, params):
16     ...
```

Why this is useful

- ▶ Each new experiment is one function.
- ▶ The common result format is reused: panels, metrics, notes.
- ▶ Noise-cleaning and FFT operations can be added without rewriting the pipeline.

Current features vs next milestones

Working now

- ✓ Image-to-array normalization
- ✓ 2D convolution engine
- ✓ Kernel presets and custom kernels
- ✓ Boundary modes
- ✓ Nearest and bilinear resizing
- ✓ Anti-aliasing comparison
- ✓ MSE, PSNR, SSIM and difference maps

Next natural extension

- ▶ Gaussian noise and salt-pepper noise UI connection
- ▶ Mean, median and Gaussian restoration experiment
- ▶ Frequency-domain filtering / FFT module
- ▶ Full before-after report export

From pixels to signal operations

Convolution, sampling, low-pass filtering and metrics - implemented with NumPy

NumPy DSP core

Beamer + TikZ

CSE 220

Thank you